

A JAX-Native Differentiable, GPU-Native Astrodynamics Substrate

*Layered Implementation Roadmap from Primitives to Operational
Propagator*

LATEX → PDF · ~21 PAGES

[DOWNLOAD PDF](#)

Reconstruction notice. This page is a summary rebuilt from fragments of the original conversation. The linked PDF is a PDF render of *this reconstruction*, not the original monograph. To replace it with the full source, drop the real `.pdf` into `/pdfs/` using the same filename.

§ Abstract

This monograph specifies a complete implementation roadmap for a high-fidelity differentiable orbit propagator built on the JAX scientific computing stack, structured as a layered architecture from the lowest computational primitives upward to a production-grade propagator with full force-model fidelity, end-to-end automatic differentiation, GPU-batched parallelism, and operational interoperability.

Each layer is treated as a self-contained engineering deliverable with explicit acceptance criteria, validation procedures, dependencies on lower layers, and potential off-ramps. The intended audience is the technical team that would actually implement this work; the document serves both as a planning artifact for resource allocation and as a reference

architecture for the implementation phase. The structure mirrors an analogous Mojo-based roadmap (No. 04) for direct comparison; identical sections in the two documents address the same engineering problem in their respective stacks.

§ Architectural premise

The substrate is structured as ten layers, each a coherent engineering deliverable with its own acceptance criteria. Below each section heading the roadmap states (1) what is built, (2) what it depends on from below, (3) what exists in the JAX ecosystem that can be borrowed, (4) where the engineering risk concentrates, and (5) the off-ramp if the layer cannot be made to work as specified.

The structural advantage of the JAX stack is that several layers — automatic differentiation, batched compilation via `vmap` and `jit`, ODE integration via Difffrax, the optimizer ecosystem in Optax — exist in production-grade form on day one. The shape of the project is "compose existing components with carefully-specified glue," not "build the foundations."

§ What's borrowed from the ecosystem

- **JAX core** — tracing, JIT, autograd primitives.
- **Difffrax** — adaptive RK methods, implicit solvers, dense output, event detection.
- **Optax** — gradient-based optimizers for downstream tasking and maneuver-planning loops.
- **jaxsgp4** — reference batched SGP4 for parity checks against operational TLE-driven tooling.
- **brandeis-orbit / heyoka.py** — reference high-fidelity propagators to anchor against during validation.

§ Risk concentrations

The roadmap calls out three layers where JAX-specific friction is unavoidable:

1. **Adaptive stepping under tracing.** Difffrax already solves most of this, but custom solvers (e.g., a Gauss-Jackson port) re-open the design tension.
2. **Force-model fidelity vs. compile time.** EGM2008 to high degree inside a `jit` trace produces enormous compute graphs; staging strategies (`static_argnums` , sharded force decomposition) are required.
3. **Reverse-mode through long integrations.** Memory pressure on the backward pass; the roadmap specifies optimal checkpointing thresholds.



§ Status & provenance

Companion to Nos. 02 and 04. Written immediately after the SPEPH monograph in conversation [2308b2c5](#). The Mojo equivalent (No. 04) is structurally parallel — identical section numbering, different stack.